

PROPIEDAD DIGITAL

Bitacora Inmutable de Propiedades en Avalanche

Workpath de Desarrollo · 36 horas

Resumen del Proyecto

El mercado inmobiliario mexicano esta afectado por desconfianza, burocracia y comisiones excesivas. Esta plataforma transforma cada inmueble en un activo inteligente mediante una L1 de Avalanche, creando una trazabilidad total y publica del historial de cada propiedad desde su construccion hasta su ultima transaccion.

Propuesta de valor central

- Bitacora inmutable: historial verificable de cada propiedad en blockchain
- Escrow inteligente: compra-venta sin intermediarios, fondos liberados solo al cumplir condiciones legales
- Tokenizacion fraccional: ladrillos (ERC-20) que democratizan la inversion inmobiliaria
- FIBRAS tokenizadas: portafolios de propiedades con distribucion automatica de dividendos

Segmentos de entrada

- Preventas con Desarrolladoras: historial limpio desde el origen, adoptado por constructoras
- Certificacion de Adeudos: validacion inmutable de predial, agua y luz
- Crowdfunding inmobiliario: acceso minorista a FIBRAS mediante tokens de participacion

Modelo de negocio

- Marketplace con comision por transaccion de inmuebles y tokens
- Certificacion premium para propiedades con historial blockchain verificado
- Acceso DeFi: comision por uso de data inmutable como colateral hipotecario en Avalanche

Stack Tecnologico

Herramienta	Funcion en el proyecto	Criterio Avalanche
Avalanche CLI	Crear y desplegar L1 local personalizada con EVM	Avalanche L1s ***
Remix IDE	Compilar y desplegar contratos en Fuji Testnet	Fuji C-Chain
Core Wallet	Autenticacion, firma de tx, custodia de tokens	Avalanche nativo **
Chainlink Functions	Verificar adeudos externos desde on-chain	Oracle en Avalanche

OpenZeppelin Wizard	Base segura para ERC-721 y ERC-20	Estandar DeFi
Snowtrace (Fuji)	Verificar y auditar contratos desplegados	Explorer Avalanche
Avalanche ICM	Mensajería cross-chain entre L1 propia y C-Chain	Cross-Chain ***
GitHub Actions	CI con lint de Solidity (solhint)	DevOps

Workpath: Fases de Desarrollo

El desarrollo se divide en 5 fases ejecutables en aproximadamente 36 horas. Cada fase incluye tareas accionables, criterios de Avalanche y entregables concretos.

Fase 1 · Setup & Arquitectura base Horas 0-4	
1	Instalar Avalanche CLI y crear L1 local (subnet personalizada con EVM compatible)
2	Configurar Core Wallet con Fuji Testnet y fondear con AVAX desde el faucet oficial
3	Conectar Remix IDE a la red Fuji (Custom RPC: https://api.avax-test.network/ext/bc/C/rpc)
4	Diseñar el modelo de datos: PropertyRecord, OwnerHistory, DebtCertificate, PropertyToken
Stack: Avalanche CLI · Fuji Testnet · Core Wallet · Remix IDE	
Avalanche: Crear la L1 local con 'avalanche network create' es el primer diferenciador técnico. Demuestra uso real de Avalanche L1s desde el inicio.	
Entregable: L1 local corriendo + Core Wallet conectado a Fuji + arquitectura documentada en README.md	

Fase 2 · Smart Contract: BitacoralInmueble (ERC-721) Horas 4-10	
1	Generar base con OpenZeppelin Wizard: ERC-721 + Ownable + URI Storage
2	Agregar struct PropertyRecord: address, constructionDate, area, cadastralKey, currentOwner
3	Implementar función registerEvent() para remodelaciones, inspecciones y cambios de dueño
4	Emitir eventos on-chain: PropertyRegistered, OwnershipTransferred, EventLogged
5	Desplegar en Fuji y verificar contrato en Snowtrace (Fuji explorer)
Stack: Solidity · ERC-721 · OpenZeppelin Wizard · Snowtrace Fuji	
Avalanche: Cada NFT representa un inmueble único. La metadata on-chain es el curriculum inmutable de la propiedad. Desplegar en Fuji demuestra integración real con Avalanche C-Chain.	
Entregable: BitacoralInmueble.sol desplegado en Fuji · Address verificada en Snowtrace · ABI exportado para el frontend	

Fase 3 · Smart Contract: Escrow + Compra-Venta Horas 10-18

- 1 Crear PropertyEscrow.sol: deposito de AVAX, condiciones de liberacion y plazo con deadline
- 2 Implementar maquina de estados: PENDING -> FUNDED -> CONDITIONS_MET -> RELEASED / REFUNDED
- 3 Integrar Chainlink Functions para verificar adeudos externos (predial, agua) via HTTP callback
- 4 Conectar escrow con BitacoraInmueble: transferencia atomica del NFT al liberar los fondos
- 5 Tests basicos en Remix: happy path completo + caso de reembolso por condicion no cumplida

Stack: Chainlink Functions · Solidity · AVAX nativo · Maquina de estados

Avalanche: Chainlink Functions corre on-chain sobre Avalanche C-Chain. La combinacion Escrow + NFT transfer atomica es el diferenciador tecnico clave vs. plataformas web2.

Entregable: PropertyEscrow.sol desplegado · flujo compra-venta end-to-end demostrable en Fuji · diagrama de estados incluido

Fase 4 · Tokenizacion: LadrilloBrick (ERC-20) + FIBRAs + ICM Horas 18-26

- 1 Crear LadrilloBrick.sol: ERC-20 fraccional vinculado a un PropertyToken (ERC-721)
- 2 Implementar dividendRelease(): distribuir rentas a holders proporcionalmente via push payment
- 3 Crear FibraManager.sol: agrupar multiples propiedades, emitir tokens de portafolio
- 4 Desplegar Cross-Chain con Avalanche ICM: enviar estado de propiedad desde L1 local a Fuji C-Chain
- 5 Frontend minimo (HTML + ethers.js) con Core Wallet integrado para mostrar balance de Ladrillos

Stack: ERC-20 · ICM Cross-Chain · Avalanche L1 · ethers.js · Core Wallet

Avalanche: MAXIMO puntaje tecnico: ICM (Interchain Messaging) es el componente mas especifico de Avalanche, sin equivalente en Ethereum ni L2s. Un mensaje cross-chain entre L1 y C-Chain demuestra dominio real del ecosistema.

Entregable: LadrilloBrick.sol + FibraManager.sol desplegados · mensaje ICM ejecutado y visible en explorer · UI conectada con Core Wallet

Fase 5 · Demo, Documentacion & Pitch Horas 26-36

- 1 Grabar flujo completo: registrar propiedad -> comprar con escrow -> recibir Ladrillos -> dividendos
- 2 Documentar en README: addresses de contratos en Fuji, arquitectura, flujos y links a Snowtrace
- 3 Preparar diagrama de arquitectura: L1 -> C-Chain con ICM + flujo de usuario end-to-end
- 4 Subir a GitHub con GitHub Actions configurado para lint de Solidity (solhint basico)

5 Preparar argumentos de modelo de negocio: comision, certificacion, acceso DeFi a data

Stack: GitHub Actions · Snowtrace · README · Pitch deck

Avalanche: En el pitch: enfatiza que ICM es exclusivo de Avalanche. L1 propia + ICM cross-chain + escrow atomico son los tres diferenciadores tecnicos no negociables.

Entregable: Repo publico en GitHub · README completo · demo grabado o en vivo · contratos verificados en Snowtrace Fuji

Arquitectura de Contratos

Los smart contracts se organizan en tres capas interconectadas dentro del ecosistema Avalanche:

Capa 1: Identidad BitacoraInmueble.sol ERC-721 · Avalanche C-Chain	Capa 2: Transaccion PropertyEscrow.sol Chainlink Functions · AVAX	Capa 3: Inversion LadrilloBrick + FibraManager ERC-20 · ICM Cross-Chain
---	--	--

Componente Critico: Avalanche ICM

Avalanche Interchain Messaging (ICM) es el diferenciador tecnico no negociable de este proyecto. A diferencia de Ethereum y sus L2s, Avalanche permite mensajeria nativa entre subredes sin puentes externos ni custodios de terceros.

Flujo ICM en el proyecto:

- La L1 personalizada registra el estado de una propiedad (owner, historial, condiciones)
- ICM transmite ese estado de forma verificable hacia la Avalanche C-Chain (Fuji)
- Los contratos en C-Chain (Escrow, LadrilloBrick) consumen esa informacion para tomar decisiones
- Resultado: sistema multi-red donde la propiedad vive en la L1 y el dinero en C-Chain

Argumentos de Pitch

L1 propia en Avalanche	Control total sobre fees, throughput y reglas de validacion. La propiedad tiene su propia cadena.
ICM Cross-Chain	Exclusivo de Avalanche. Sin puentes externos. Los datos de propiedad fluyen entre redes de forma nativa.
Escrow atomico + Chainlink	El dinero solo se mueve cuando las condiciones legales y on-chain se cumplen simultaneamente.
Tokenizacion ERC-20 (Ladrillos)	Desde \$50 USD se puede invertir en bienes raices fraccionales con dividendos automaticos.
Reduccion de costos reales	De meses a dias. De 5-10% de comision a una fraccion. Verificacion instantanea vs. semanas de burocracia.